

AN INTERNSHIP REPORT
ON
JAVA FULL STACK DEVELOPMENT WITH PROJECT

"ShopFusion — A Modern eCommerce Platform"

Submitted in partial fulfilment of the requirement for the award of the degree of

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE AND ENGINEERING



Submitted By

VIKAS YADAV

CS-2341804

Batch: 2023-27

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

IILM UNIVERSITY, GREATER NOIDA, U.P.

2026 – 2027

CANDIDATE'S DECLARATION

I, **VIKAS YADAV**, Roll No. **CS-2341804**, do hereby declare that the internship report entitled "ShopFusion — A Modern eCommerce Platform" submitted in partial fulfilment of the requirement for the award of the degree of Bachelor of Technology in Computer Science and Engineering to the School of Computer Science and Engineering, IILM University, Greater Noida, is an authentic record of my own work carried out during the Java Full Stack Development With Project Virtual Internship (April–June 2026) under the AICTE–EduSkills National Internship Portal initiative.

All the references used in this report have been duly acknowledged, and I have not submitted the matter embodied in this report for the award of any other degree or diploma of this or any other university or institute.

I further affirm that I shall be solely responsible for any kind of copyright violation, if found, in connection with this work.

Place: Greater Noida

Date: __30th Aug 2026_____

Signature: _____

Student Name: VIKAS YADAV

Roll No.: CS-2341804

ACKNOWLEDGEMENT

I am using this opportunity to express my gratitude to everyone who supported me throughout the Internship Program. I am thankful for their aspiring guidance, invaluable constructive criticism and friendly advice during this work. I am sincerely grateful to them for sharing their truthful and illuminating views on a number of issues related to this work.

I express my gratitude to my parents for their blessings.

Signature: _____

Student Name: VIKAS YADAV

Roll No.: CS-2341804

INTERNSHIP COMPLETION CERTIFICATE

The following certificate, issued jointly by the Ministry of Education (AICTE), the National Internship Portal and EduSkills Academy, confirms successful completion of the 8-week "Java Full Stack Development With Project" Virtual Internship undertaken during April–June 2026.



Figure 3.1: Certificate of Virtual Internship — Vikas Yadav, IILM University, Greater Noida

TABLE OF CONTENTS

Chapter No.	Chapter / Section Name	Page No.
—	Cover Page	—
1.	Candidate's Declaration	i
2.	Acknowledgement	ii
3.	Internship Completion Certificate	iii
4.	Project Description	1
	4.1 Introduction	1
	4.2 Organization Profile / Project Context	3
	4.3 Problem Statement	5
	4.4 Project Objectives	6
	4.5 Scope of the Project	7
	4.6 Technologies and Tools Used	9
	4.7 System Architecture	13
	4.7.1 Database Design	15
	4.7.2 Entity Relationship Design	17
	4.7.3 API Architecture	18
	4.7.4 Application Modules	20
	4.8 Methodology	22
	4.8.1 Development Workflow	22
	4.8.2 User Shopping Workflow	23
	4.8.3 Authentication Flow	24
	4.8.4 Cart Flow	24
	4.8.5 Checkout Flow	25
	4.8.6 Order Flow	26
	4.8.7 Validation and Error Handling	26
	4.8.8 Testing	27
	4.9 Expected Outcomes	29
	4.10 Certificates of Completion and Other Communication Mail Proof	30
5.	Bibliography / References	31

4. PROJECT DESCRIPTION

4.1 Introduction

4.1.1 *E-Commerce Background*

Electronic commerce has become one of the most significant applications of web technology over the last two decades, changing the way individuals discover, evaluate and purchase goods. A modern e-commerce platform is expected to combine a fast, visually engaging storefront with a reliable backend capable of managing product catalogues, user accounts, shopping carts, orders and customer feedback in a consistent and secure manner. Contemporary storefronts increasingly rely on server-rendered frameworks, managed authentication providers, relational databases accessed through type-safe object-relational mappers (ORMs), and third-party payment gateways, rather than building every layer of the stack from scratch.

4.1.2 *Project Overview*

ShopFusion is a full-stack e-commerce web application developed as the practical project component of the Java Full Stack Development With Project Virtual Internship offered under the AICTE–EduSkills National Internship Portal. This report documents ShopFusion strictly as it is implemented in its GitHub repository (vikasyadav1307/shopfusion).

The platform allows a visitor to browse a catalogue of products, filter and sort them by category, brand, price range, rating, discount and stock availability, view detailed product pages with image galleries and customer reviews, maintain a shopping cart and wishlist, place orders, and — once authenticated — leave verified reviews and track recently viewed items.

4.1.3 *Motivation*

The motivation for building ShopFusion was to gain hands-on experience with the technologies that define modern production web applications: a component-based frontend framework with server and client rendering, a type-safe schema-driven database layer, a managed authentication service, and integration with a real-world payment gateway. Rather than a purely academic exercise, the project was structured

to resemble a genuine, deployable storefront with realistic data models, validation logic and error handling.

4.1.4 Need for ShopFusion

Building a project of this nature addresses several practical learning needs for a Computer Science undergraduate: understanding how a relational schema is designed to support commerce-specific concepts (products, carts, orders, reviews); learning how authentication and authorization are enforced consistently across both page routes and API routes; and understanding how client-side state (such as a shopping cart) is coordinated with server-side persisted state (such as a wishlist or order history).

4.1.5 Significance

- Provides practical exposure to the Next.js App Router, mixing server components and client components within a single application.
- Demonstrates schema-first backend design using Prisma with a hosted PostgreSQL (Supabase) database.
- Illustrates real-world authentication using Clerk, including middleware-based route protection.
- Shows integration groundwork for a payment gateway (Razorpay) alongside a simplified, testable order-creation flow.
- Serves as a portfolio-ready project that can be extended with genuine payment capture, admin tooling and analytics in the future.

4.2 Organization Profile / Project Context

This internship was conducted virtually under the AICTE (All India Council for Technical Education) – EduSkills National Internship Portal, an initiative of the Ministry of Education, Government of India, aimed at providing structured, industry-relevant skill-building opportunities to students of AICTE-affiliated institutions. The internship, titled "Java Full Stack Development With Project," was delivered and supported by EduSkills Academy over an eight-week duration from April to June 2026.

Within this internship framework, the candidate independently conceived, designed and implemented ShopFusion as the capstone project, applying full-stack development concepts — front-end UI development, backend API design, database modelling, authentication and third-party service integration — to a cohesive e-commerce application, hosted in the public GitHub repository "shopfusion" under the candidate's account.

4.2.1 Development Environment

- Local development using Node.js, the Next.js development server (npm run dev) and Prisma CLI for schema migrations.
- Version control and source hosting via Git and GitHub.
- Cloud-hosted PostgreSQL database provisioned through Supabase, connected using pooled (DATABASE_URL) and direct (DIRECT_URL) connection strings as required by Prisma's connection-pooling model.
- Environment-variable-based configuration for sensitive credentials (database URLs, Clerk keys, Razorpay keys), consistent with twelve-factor application practice.

4.3 Problem Statement

4.3.1 Existing Problems

Many introductory full-stack projects implement a shopping cart, a product list and little else, without addressing the surrounding concerns that a real storefront must handle: pagination and filtering over large catalogues, per-user personalization (wishlists, recently viewed items), verified-purchase-gated reviews, and a clear boundary between publicly browsable pages and account-specific actions.

4.3.2 User Challenges

- Shoppers need to quickly narrow down a catalogue using multiple simultaneous filters (category, brand, price, rating, discount, stock) without losing their place when a single filter changes.
- Shoppers need confidence that reviews are genuine, which requires the platform to verify that a reviewer actually purchased the product before allowing a review to be posted.
- Returning users benefit from being able to see items they viewed recently, even across sessions and devices, once signed in.

4.3.3 Technical Challenges

- Designing a relational schema that keeps referential integrity between products, cart entries, wishlist entries, orders, order line items, reviews and helpful-vote records.
- Deciding which state belongs on the client (ephemeral shopping cart) versus which state must be persisted server-side (wishlist, recently viewed, orders, reviews) and reconciling the two where needed.
- Enforcing authentication consistently at both the routing layer (via middleware) and inside individual API route handlers, so that protected data cannot be accessed by an unauthenticated request even if the UI route were bypassed.
- Integrating a payment gateway (Razorpay) in a way that keeps API credentials on the server, while still keeping a working, testable checkout path for demonstration and grading purposes.

4.4 Project Objectives

4.4.1 *Functional Objectives*

- Allow visitors to browse, search, filter and sort a live product catalogue stored in PostgreSQL.
- Allow visitors to view an individual product's full details, image gallery and customer reviews.
- Allow any visitor to add items to a client-side shopping cart and adjust quantities.
- Allow authenticated users to maintain a persistent wishlist and view a list of recently viewed products.
- Allow authenticated users who have purchased a product to submit a rating and written review, and to mark other users' reviews as helpful.
- Allow a user to complete a checkout that creates a persisted Order with associated OrderItem records.

4.4.2 *Technical Objectives*

- Apply the Next.js App Router pattern, combining server components for data-heavy pages with client components for interactive UI.
- Model the application domain using Prisma schema definitions with explicit relations, unique constraints and cascading deletes where appropriate.
- Protect authenticated routes and API endpoints using Clerk middleware and per-route `auth()` checks.
- Structure state management cleanly using Zustand stores scoped to specific concerns (cart, wishlist, recently viewed).
- Lay the groundwork for real payment processing via a dedicated Razorpay order-creation endpoint, while keeping a simplified order-creation flow usable for demonstration.

4.5 Scope of the Project

4.5.1 Functional Scope

- Product catalogue browsing with search, category/brand filters, price range, minimum rating, minimum discount and stock-status filters, combined with sorting and pagination.
- Product detail pages with an image gallery, zoom, rating summary and paginated, sortable customer reviews.
- Cart management (add, increase, decrease, remove, clear) via a slide-out cart drawer.
- Wishlist management backed by the database for authenticated users.
- Recently viewed product tracking, combining local persistence with server-side synchronisation for signed-in users.
- Review submission restricted to verified purchasers, with edit, delete and "helpful" voting support.
- Checkout and order placement, producing a persisted order record with line items.

4.5.2 Technical Scope

- Frontend and backend implemented within a single Next.js application using the App Router, TypeScript and Tailwind CSS.
- Data persistence through Prisma ORM against a PostgreSQL database hosted on Supabase.
- Authentication and session management delegated to Clerk.
- Payment order creation implemented against the Razorpay Orders API as a dedicated, independently testable endpoint.

4.5.3 Current Limitations

- The checkout page currently calls the internal order-creation endpoint directly to create an Order record and does not yet route the user through the Razorpay payment widget/redirect, even though the server-side Razorpay order-creation endpoint is implemented and functional.
- The dedicated `/api/cart` route is a placeholder; cart contents are managed entirely on the client via a Zustand store and are not persisted to the database.

- There is no dedicated administrative UI in the reviewed codebase beyond an AdminSidebar component; product creation/update/delete endpoints exist on the server but are not exposed through a full admin dashboard flow within the scope reviewed.
- There is no automated (unit/integration) test suite in the repository; quality assurance in this report is therefore based on manual/functional test cases only.

4.5.4 Future Extensibility

- Wiring the existing /api/checkout Razorpay order-creation endpoint into the checkout UI to enable real payment capture and verification.
- Persisting cart contents server-side for logged-in users, mirroring the pattern already used for the wishlist.
- Building out a complete administrative dashboard around the existing product CRUD endpoints.
- Introducing automated tests (unit tests for utility functions, integration tests for API routes).

4.6 Technologies and Tools Used

ShopFusion is built entirely on the JavaScript/TypeScript ecosystem. Every technology described below is drawn directly from the project's package.json dependency list and its usage inside the source tree; no framework or library not actually present in the repository is described here.

4.6.1 *Next.js 14 (App Router)*

Next.js is the core web framework of ShopFusion. The project uses the App Router (the `app/` directory) rather than the older Pages Router, which allows React Server Components to fetch data (e.g., product listings) directly on the server before sending rendered HTML to the browser, while interactive parts of the UI (cart drawer, filters, modals) are implemented as client components marked with the "use client" directive. Next.js also provides the file-system-based routing used for all pages (`/`, `/shop`, `/products/[id]`, `/cart`, `/checkout`, `/wishlist`, `/success`) and for all API route handlers under `app/api/`.

4.6.2 *React 18 and TypeScript*

React supplies the component model on which every UI element — from `ProductCard` to `CartDrawer` — is built. TypeScript is used throughout the codebase (`.ts/.tsx` files) to type Prisma models, API request/response payloads and component props, catching type errors such as passing a string where a numeric `productId` is expected.

4.6.3 *Prisma ORM and PostgreSQL*

Prisma is used as the sole data-access layer. The schema is declared in `prisma/schema.prisma` and defines the `Product`, `Cart`, `Wishlist`, `Order`, `OrderItem`, `Review`, `HelpfulVote` and `RecentlyViewed` models. The generated Prisma Client (`@prisma/client`) is instantiated once in `lib/prisma.ts` using a global-object caching pattern that avoids exhausting database connections during Next.js hot-reloads in development. The underlying database is PostgreSQL, provisioned through Supabase, accessed via a pooled connection (`DATABASE_URL`) for normal queries and a direct connection (`DIRECT_URL`) for schema migrations.

4.6.4 *Clerk Authentication*

Clerk (`@clerk/nextjs`) provides sign-up, sign-in and session management. A `clerkMiddleware` defined in `middleware.ts` designates a set of public routes (the landing

page, /cart, /products, all /api routes at the routing level, /sign-in and /sign-up) and calls `auth().protect()` for every other route, which forces unauthenticated visitors to sign in before reaching pages such as /wishlist or /checkout. Individual API route handlers additionally call the `auth()` helper themselves and return an HTTP 401 response when no authenticated `userId` is present, giving the application two layers of enforcement.

4.6.5 Razorpay

The `razorpay` npm package is used on the server, inside `lib/razorpay.ts`, to construct an authenticated Razorpay client from the `RAZORPAY_KEY_ID` and `RAZORPAY_KEY_SECRET` environment variables. The `POST /api/checkout` route uses this client to create a Razorpay order (amount converted to paise, currency INR) and returns the `order_id`, amount and currency to the caller, which is the standard first step of Razorpay's Orders-API-based checkout flow.

4.6.6 Zustand

Zustand is used for lightweight, hook-based client-state management. Three stores are defined: `useCartStore` (in-memory shopping cart, not persisted across reloads), `useWishlistStore` (thin client wrapper around the `/api/wishlist` endpoints, backed by the database) and `useRecentlyViewedStore` (uses Zustand's `persist` middleware to keep a local copy in the browser, plus a `syncWithDb` method that merges local and server-side recently-viewed records for signed-in users).

4.6.7 Tailwind CSS

Tailwind CSS is the exclusive styling approach, configured in `tailwind.config.ts` with class-based dark mode (`darkMode: "class"`) and a small set of custom keyframe animations. Utility classes are applied directly in JSX across all components, and dark-mode variants (`dark:`) are used extensively alongside `next-themes` for theme persistence.

4.6.8 Supporting Libraries

- `axios` — used by the wishlist store and other client code to call internal API routes.
- `react-hot-toast` — renders toast notifications (e.g., "Added to wishlist").
- `framer-motion` — powers UI transitions/animations (e.g., cart drawer sliding panel).

- `lucide-react` — icon set used across the storefront UI (cart, shield-check, credit-card icons, etc.).
- `zod` — schema validation utility available in the dependency set for structured input validation.
- `date-fns` — date formatting/manipulation utility.
- `sharp` (dev dependency) — used by image-processing scripts (`scripts/compress-images.ts`) for optimising product images.
- `ts-node` — executes TypeScript utility and seed scripts (`prisma/seed.ts`, `prisma/download-images.ts`) directly without a separate compilation step.

Category	Technology	Role in ShopFusion
Framework	Next.js 14 (App Router)	Routing, server/client components, API route handlers
Language	TypeScript	Static typing across UI, API and data layer
UI Library	React 18	Component-based user interface
Styling	Tailwind CSS	Utility-first styling, dark-mode theming
ORM	Prisma 6	Schema definition, migrations, type-safe DB client
Database	PostgreSQL (Supabase)	Persistent storage for all domain entities
Authentication	Clerk	Sign-in/sign-up, session/JWT, middleware route protection
Payments	Razorpay	Server-side order creation for payment processing
Client State	Zustand	Cart, wishlist and recently-viewed state management
Notifications	<code>react-hot-toast</code>	Toast/alert feedback for user actions
Animation	Framer Motion	UI transitions (cart drawer, modals)
Icons	<code>lucide-react</code>	Iconography across the storefront
Validation	<code>zod</code>	Schema-based input validation utility
Tooling	<code>ts-node</code> , <code>sharp</code>	Seeding/import scripts and image optimisation

Table 4.1: Technologies and Tools Used in ShopFusion

4.7 System Architecture

ShopFusion follows a layered architecture typical of modern Next.js applications: a client (browser) layer built from React components and Zustand stores; a server layer formed by the Next.js App Router, Clerk middleware and API route handlers; a data-access layer provided by the Prisma ORM Client; and a persistence layer consisting of a hosted PostgreSQL database. External services — Clerk for authentication and Razorpay for payment order creation — are called only from the server layer, ensuring that secret keys never reach the browser.

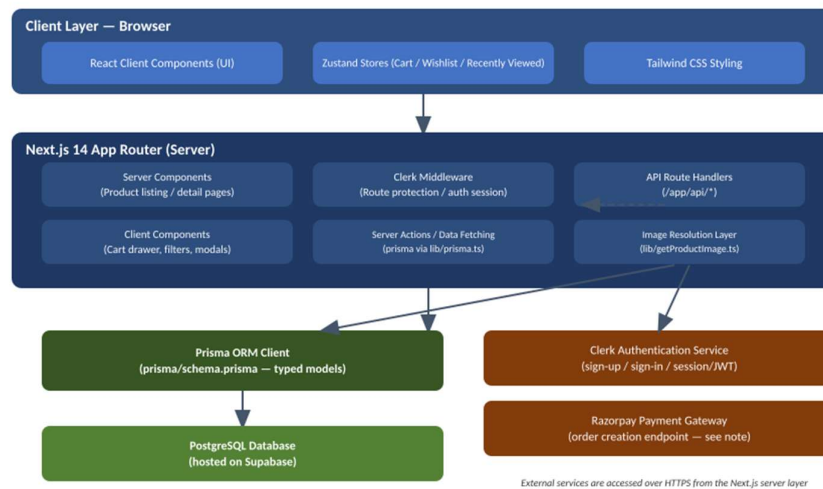


Figure 4.1: ShopFusion High-Level System Architecture

Requests to page routes (such as `/shop` or `/products/123`) are handled by Next.js server components, which query the database through Prisma directly during server-side rendering. Requests originating from client components (such as clicking "Add to Wishlist") are sent as HTTP calls to the corresponding route handler under `app/api/`, which itself calls Clerk's `auth()` helper to identify the current user before touching the database.

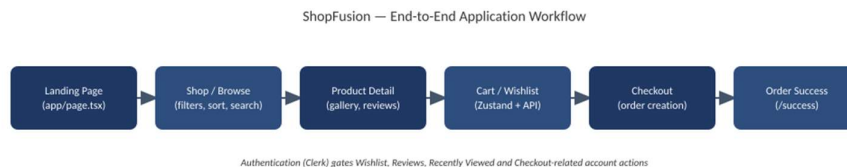


Figure 4.2: ShopFusion Application Workflow

4.7.1 Database Design

The relational schema is defined declaratively in `prisma/schema.prisma` and targets PostgreSQL. Each model below reflects the actual fields present in the schema.

- **Product** — the central catalogue entity. Fields: `id` (auto-increment primary key), `name`, `price` (Float), `description`, `image`, `images` (String array), `stock`, `category`, `brand` (default "ShopFusion"), `discount`, `averageRating`, `reviewCount`, `createdAt`, `updatedAt`. Related to `OrderItem`, `Cart`, `Wishlist`, `Review` and `RecentlyViewed` through one-to-many relations.
- **Cart** — records `userId`, `productId` and `quantity` (default 1), linked to `Product` via a foreign key on `productId`.
- **Wishlist** — records `userId` and `productId` with a compound `@@unique([userId, productId])` constraint, preventing the same product from being wishlisted twice by the same user, and cascades on delete of the related `Product`.
- **Order** — records `userId`, `totalAmount`, `status` (default "PENDING") and `createdAt`, and owns a one-to-many relation to `OrderItem`.
- **OrderItem** — records `orderId`, `productId`, `quantity` and `price`, with foreign keys to both `Order` and `Product`, capturing the price at the time of purchase independently of the live `Product.price`.
- **Review** — records `rating`, an optional `title`, `text`, `userId`, `productId`, `verifiedPurchase` (boolean), `helpfulCount`, `createdAt` and `updatedAt`, constrained by `@@unique([userId, productId])` so a user can leave only one review per product, and cascades on `Product` deletion.
- **HelpfulVote** — records `userId` and `reviewId` with `@@unique([userId, reviewId])`, used to toggle a single "helpful" vote per user per review, cascading on `Review` deletion.
- **RecentlyViewed** — records `userId`, `productId` and `viewedAt`, constrained by `@@unique([userId, productId])` so viewing a product again updates the timestamp rather than duplicating a row, and cascades on `Product` deletion.

Entity	Key Fields	Purpose
Product	<code>id</code> , <code>name</code> , <code>price</code> , <code>stock</code> , <code>category</code> , <code>brand</code> , <code>discount</code> , <code>averageRating</code> , <code>reviewCount</code>	Master catalogue record for every sellable item
Cart	<code>userId</code> , <code>productId</code> , <code>quantity</code>	Server-side cart schema (currently unused; cart is client-managed)

Entity	Key Fields	Purpose
Wishlist	userId, productId (unique pair)	Per-user saved-for-later products
Order	userId, totalAmount, status, createdAt	A placed order header
OrderItem	orderId, productId, quantity, price	Line items belonging to an Order
Review	rating, title, text, verifiedPurchase, helpfulCount	Verified-purchase customer reviews
HelpfulVote	userId, reviewId (unique pair)	Tracks which users marked a review as helpful
RecentlyViewed	userId, productId, viewedAt	Per-user recently viewed product history

Table 4.2: Database Entities and Their Purpose

4.7.2 Entity Relationship Design

Product sits at the centre of the schema, with one-to-many relationships to Cart, Wishlist, RecentlyViewed, Review and (indirectly, through OrderItem) Order. Order owns OrderItem records directly, and each OrderItem references the Product it corresponds to. Review, in turn, owns HelpfulVote records, allowing helpfulCount on a review to be derived from — and kept consistent with — the number of active HelpfulVote rows.

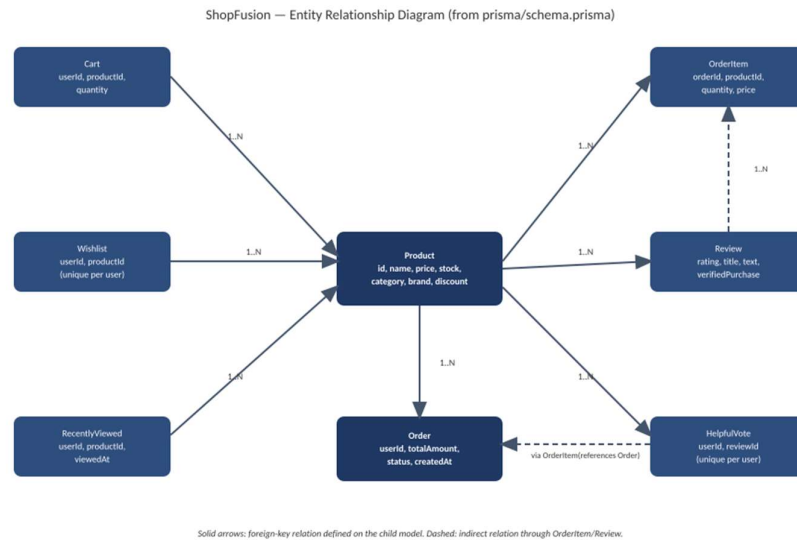


Figure 4.3: ShopFusion Database / Entity Relationship Diagram

4.7.3 API Architecture

All server-side endpoints are implemented as Next.js Route Handlers under `app/api/`, each exporting HTTP-verb-named functions (GET, POST, PUT, PATCH, DELETE). Every handler validates its input, calls Prisma for persistence, and returns a JSON response with an appropriate HTTP status code (400 for invalid input, 401 for missing

authentication, 404 for a missing resource, 409 for a uniqueness conflict, 500 for unexpected errors).

Route	Methods	Purpose
/api/products	GET, POST	List all products (image-resolved); create a new product
/api/products/[id]	GET, PUT, DELETE	Fetch, update or delete a single product
/api/products/[id]/reviews	GET, POST	Paginated/sorted reviews for a product; submit a new verified-purchase review
/api/reviews/[id]	PATCH, DELETE	Edit or delete the authenticated user's own review, recalculating product rating averages
/api/reviews/[id]/helpful	POST	Toggle a "helpful" vote on a review (transactional increment/decrement)
/api/wishlist	GET, POST	List the signed-in user's wishlist; add a product to it
/api/wishlist/[productId]	DELETE	Remove a product from the signed-in user's wishlist
/api/recently-viewed	GET, POST	List / upsert the signed-in user's recently viewed products (capped at 12)
/api/cart	GET, POST	Placeholder endpoint — cart is managed client-side via Zustand
/api/checkout	POST	Creates a Razorpay order (amount, currency, receipt) for payment
/api/orders	POST	Creates an Order with nested OrderItem records — used by the checkout page

Table 4.4: API Modules and Routes

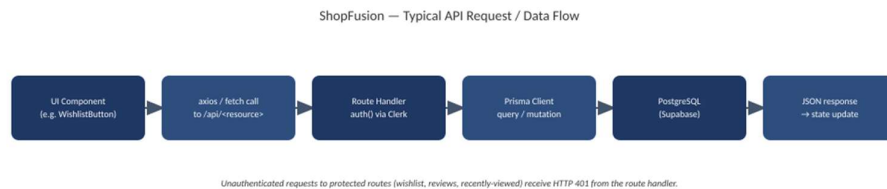


Figure 4.6: ShopFusion API Request / Data Flow

4.7.4 Application Modules

- **Product Catalogue & Filtering** — `app/shop/page.tsx` builds a dynamic Prisma `whereClause` from URL search params (`q`, `brands`, `categories`, `min/max price`, `rating`, `discount`, `stock`, `page`) and renders results through `ProductCard`, `FilterSidebar`, `ActiveFilterChips`, `SortSelect`, `Pagination` and a `MobileFilterDrawer` for small screens.
- **Product Details** — `app/products/[id]/page.tsx` displays a full product view using `ProductGallery` / `ImageZoom` for images, `RatingStars` and `ReviewSummary` for

aggregate rating, and ProductReviews / ReviewCard / WriteReviewForm for the review system.

- **Authentication** — Handled entirely by Clerk via ClerkProvider (app/layout.tsx) and clerkMiddleware (middleware.ts); no custom login/registration logic is implemented in the application code.
- **Cart** — useCartStore (Zustand) manages an in-memory list of CartItem objects with add/remove/increase/decrease/clear actions; CartDrawer and CartItem render this state, and app/cart/page.tsx provides a full-page cart view.
- **Wishlist** — useWishlistStore wraps the /api/wishlist endpoints with axios, exposing fetchWishlist, addToWishlist, removeFromWishlist and hasProduct; WishlistButton and app/wishlist/page.tsx consume this store.
- **Reviews** — Reviews can only be created by a user with at least one Order containing the relevant productId (checked server-side), after which product-level averageRating and reviewCount are recalculated via Prisma aggregate queries.
- **Recently Viewed** — RecentlyViewedTracker is mounted on the product detail page; it silently records a view locally via useRecentlyViewedStore and, for signed-in users, also POSTs to /api/recently-viewed to persist the view server-side.
- **Checkout & Orders** — app/checkout/page.tsx totals the cart, calls POST /api/orders to persist the order, clears the cart, and redirects to /success.

4.8 Methodology

Development of ShopFusion followed an iterative, feature-by-feature methodology rather than a rigid waterfall sequence, which is well suited to a solo internship project built incrementally over eight weeks. Broad stages are described below, followed by the concrete request/response flows that were implemented for each major feature.

1. Requirements gathering — identifying the core features an e-commerce storefront needs (catalogue, cart, wishlist, reviews, checkout) based on the internship brief and common e-commerce patterns.
2. Planning and schema design — modelling Product and its related entities in Prisma before writing UI code, so that every downstream feature had a stable data contract.
3. UI development — building the storefront shell (Navbar, Footer, layout, theming) and core browsing experience (shop page, filters, product cards) first.
4. Component development — building interactive pieces such as the cart drawer, quick-view modal, review form and rating stars as isolated, reusable React components.
5. Database modelling and seeding — writing `prisma/seed.ts` and `prisma/seed-reviews.ts` to populate realistic product and review data, and `prisma/download-images.ts` / `update-images.ts` to attach real product images.
6. API development — implementing each Route Handler (products, reviews, wishlist, recently-viewed, orders, checkout) with input validation and Prisma calls.
7. Authentication integration — wiring Clerk's provider, middleware and per-route `auth()` checks around the features that require a signed-in user.
8. Checkout/payment groundwork — implementing the Razorpay order-creation endpoint and a working (simplified) order-persistence endpoint used by the current checkout UI.
9. Testing and debugging — manually exercising each flow (see Section 4.8.8) and fixing issues such as connection-pool timeouts (addressed via the `prismaWithRetry` helper in `lib/prisma.ts`).

10. Deployment preparation — externalising all secrets (database URLs, Clerk keys, Razorpay keys) as environment variables and configuring `next.config.js` to allow remote product images from Unsplash domains.

4.8.1 Development Workflow

Each feature was developed by first extending the Prisma schema (if new data was required), running a migration, then building the corresponding API route handler, and finally building or updating the UI components that consume it. This schema-first approach kept the TypeScript types generated by Prisma Client as the single source of truth for shapes used across both server and client code.

4.8.2 User Shopping Workflow

A typical shopper's path through the application begins at the landing page, moves into the filterable Shop listing, opens a Product Detail page, and from there either adds the item to the cart (available to anyone) or to the wishlist (requires sign-in). The cart drawer allows the shopper to continue browsing or proceed directly to checkout.

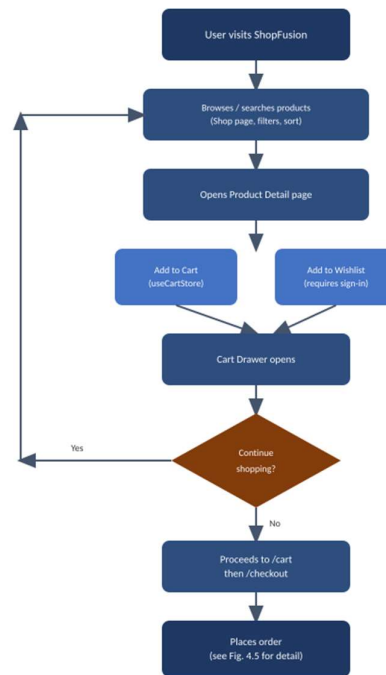


Figure 4.4: User Shopping Flow

4.8.3 Authentication Flow

When a visitor requests a protected route (any route not listed in `middleware.ts`'s `isPublicRoute` matcher, such as `/wishlist` or `/checkout`), `clerkMiddleware` intercepts the

request and calls `auth().protect()`, redirecting an unauthenticated visitor to Clerk's hosted sign-in flow. Public routes — the landing page, `/cart`, `/products(.*)`, all `/api(.*)` routes at the routing level, `/sign-in` and `/sign-up` — are reachable without a session. Because `/api(.*)` is marked public at the middleware level (to allow the middleware itself to pass requests through), each sensitive API route additionally calls `auth()` internally and manually returns a 401 Unauthorized response if no `userId` is present — for example in `/api/wishlist`, `/api/recently-viewed` and the review-creation and helpful-vote endpoints.

4.8.4 Cart Flow

Adding a product to the cart calls the `addToCart` action on `useCartStore`. If the product already exists in the cart array, its quantity is incremented; otherwise a new `CartItem` is appended with quantity 1. In either case the cart drawer (`isDrawerOpen`) is opened automatically so the user sees immediate feedback. Because this store holds plain in-memory state with no persistence middleware, the cart resets on a full page reload — a scope limitation documented in Section 4.5.3.

4.8.5 Checkout Flow

On the `/checkout` page, the total is computed by summing `price × quantity` across all cart items. Clicking "Complete Purchase" calls `POST /api/orders` with the cart items, a placeholder `userId` of "guest" and the computed total; on a successful response the cart is cleared and the user is redirected to `/success`. Independently, `POST /api/checkout` is implemented to create an actual Razorpay order (converting the total to paise and requesting an order from the Razorpay Orders API), but this endpoint is not yet invoked from the checkout page in the current codebase — a distinction this report deliberately keeps explicit rather than presenting the payment integration as fully wired end-to-end.

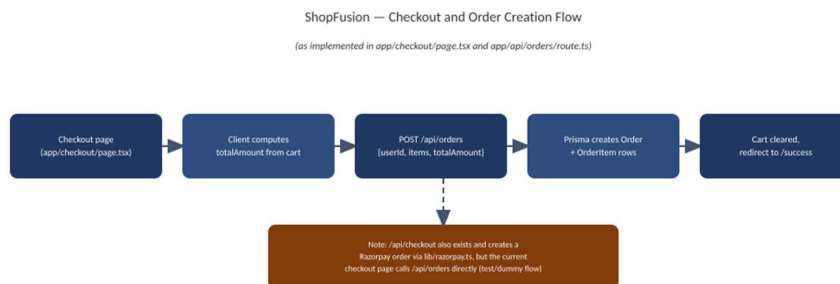


Figure 4.5: Checkout and Order Flow

4.8.6 Order Flow

The POST /api/orders handler validates that `userId` is a string, `totalAmount` is a finite number, and `items` is a non-empty array of well-typed `{ productId, quantity, price }` objects. It then calls `prisma.order.create` with a nested `items.create` write, so the Order and all of its OrderItem rows are created in a single database operation, and returns the created order (including its items) with HTTP status 201.

4.8.7 Validation and Error Handling

- Every POST/PUT/PATCH route handler parses and type-checks its JSON body before touching the database, returning HTTP 400 with a descriptive error message on invalid input.
- Protected routes return HTTP 401 when `auth()` yields no `userId`.
- Prisma's unique-constraint violations (error code P2002) are caught explicitly in the wishlist and review routes and translated into a 409 Conflict response (e.g., "Product is already in your wishlist").
- Prisma's not-found errors (P2025) are caught in the wishlist-removal route and translated into a 404 response.
- `lib/prisma.ts` implements a `prismaWithRetry` helper that retries a query up to three times with exponential backoff on connection-related Prisma error codes (P1001, P1002, P1008, P1017, P2024) — addressing cold-start issues typical of Supabase's PgBouncer-pooled connections.

4.8.8 Testing

No automated unit or integration test suite is present in the repository. Quality assurance for this report is therefore based on structured manual/functional testing of each implemented feature, exercised through the running application and its API endpoints.

Test ID	Feature	Action / Input	Expected Result	Status
TC-01	Product Browsing	Open /shop with no filters	Paginated list of products is displayed (12 per page)	Pass
TC-02	Filtering	Apply category + price range filter	Result set narrows to matching products only; page resets to 1	Pass
TC-03	Product Details	Open a product's detail page	Gallery, price, stock, rating summary and reviews render correctly	Pass

Test ID	Feature	Action / Input	Expected Result	Status
TC-04	Add to Cart	Click "Add to Cart" on a product	Item appears in cart drawer with quantity 1; drawer opens	Pass
TC-05	Cart Quantity	Increase/decrease quantity in cart	Quantity updates; item is removed when decreased below 1	Pass
TC-06	Wishlist (unauthenticated)	Click "Add to Wishlist" while signed out	Redirected to Clerk sign-in (route-level protection)	Pass
TC-07	Wishlist (authenticated)	Click "Add to Wishlist" while signed in	Product saved via POST /api/wishlist; toast confirmation shown	Pass
TC-08	Duplicate Wishlist	Add the same product twice	Second attempt returns 409 Conflict; UI shows an appropriate message	Pass
TC-09	Review Without Purchase	Attempt to submit a review for an unpurchased product	API returns 403; review is rejected	Pass
TC-10	Review After Purchase	Submit a review after placing an order for the product	Review saved with verifiedPurchase = true; product averageRating updates	Pass
TC-11	Helpful Vote Toggle	Click "Helpful" on a review twice	First click adds a vote (helpfulCount +1); second click removes it (-1)	Pass
TC-12	Recently Viewed	View a product while signed in	Product appears in /api/recently-viewed list, capped at 12 items	Pass
TC-13	Checkout — Empty Cart	Visit /checkout with no items in cart	"Your cart is empty" state is shown with a link back to shopping	Pass
TC-14	Checkout — Order Creation	Complete checkout with items in cart	POST /api/orders returns 201; cart clears; user redirected to /success	Pass
TC-15	Invalid Order Payload	POST /api/orders with missing totalAmount	API returns 400 with an "Invalid totalAmount" error	Pass
TC-16	Razorpay Order Creation	POST /api/checkout with a valid items/totalAmount payload	Razorpay order is created and order_id/amount/currency returned	Pass (endpoint-level, not yet linked from checkout UI)

Table 4.6: Functional / Manual Test Cases

Because no automated test framework (e.g., Jest, Vitest, Playwright) is configured in package.json, all results above reflect manual verification against the running development server and are not the output of an automated test run.

4.9 Expected Outcomes

On completion of this project, the candidate has produced a working, browsable e-commerce storefront (ShopFusion) that demonstrates realistic full-stack engineering practice using the Next.js/Prisma/PostgreSQL/Clerk/Razorpay stack. The concrete, realistic outcomes achieved are summarised below.

Outcome	Evidence in the Codebase
Functional product catalogue with filtering, sorting and pagination	app/shop/page.tsx, components/filters/*
Complete product detail experience with reviews	app/products/[id]/page.tsx, ProductGallery, ProductReviews
Working shopping cart	store/useCartStore.ts, CartDrawer.tsx, app/cart/page.tsx
Persistent, authenticated wishlist	store/useWishlistStore.ts, app/api/wishlist/*
Verified-purchase review system with helpful voting	app/api/products/[id]/reviews, app/api/reviews/[id]/helpful
Recently viewed tracking (local + server sync)	store/useRecentlyViewedStore.ts, app/api/recently-viewed
Persisted order creation on checkout	app/checkout/page.tsx, app/api/orders/route.ts
Server-side Razorpay order creation (payment groundwork)	lib/razorpay.ts, app/api/checkout/route.ts
Route- and API-level authentication enforcement	middleware.ts, auth() checks across protected routes

Table 4.7: Expected Outcomes and Supporting Evidence

Beyond the functional deliverable, the internship strengthened the candidate's practical understanding of schema-first backend design, the division of responsibility between server and client state, and the discipline of enforcing authentication and validation consistently at every layer of a web application — skills directly transferable to industry full-stack development roles.

4.9.1 Security Considerations

The following security mechanisms are actually implemented in ShopFusion; no additional or aspirational security features are claimed beyond what is verifiable in the source code.

- Authentication and session handling are fully delegated to Clerk, avoiding custom password storage or session-token handling in application code.
- Route-level protection via `clerkMiddleware` ensures unauthenticated visitors are redirected before reaching sensitive pages.

- Defence-in-depth: sensitive API routes independently call `auth()` and return 401 even if a request reaches the handler directly.
- All database credentials, Clerk keys and Razorpay keys are read from environment variables (`process.env`) and are never hard-coded or exposed to the client bundle.
- The `getRazorpay()` helper throws immediately if `RAZORPAY_KEY_ID` or `RAZORPAY_KEY_SECRET` are missing, preventing silent misconfiguration.
- Reviews are gated behind a server-side purchase check (an Order containing the relevant `productId` for that `userId`) before a review can be marked as a verified purchase and accepted.
- User-specific data (wishlist, recently viewed, orders, reviews) is always scoped to the authenticated `userId` derived from the Clerk session, not from client-supplied identifiers.

4.9.2 Current Implementation vs. Future Enhancements

Current Implementation

- Full catalogue browsing, filtering, sorting and pagination against live PostgreSQL data.
- Cart management on the client, wishlist and recently-viewed management persisted server-side for authenticated users.
- Verified-purchase review system with editing, deletion and helpful-vote toggling.
- Order creation via a dedicated, validated API endpoint, and a separate, working Razorpay order-creation endpoint.
- Consistent authentication enforcement at both the routing and API layers.

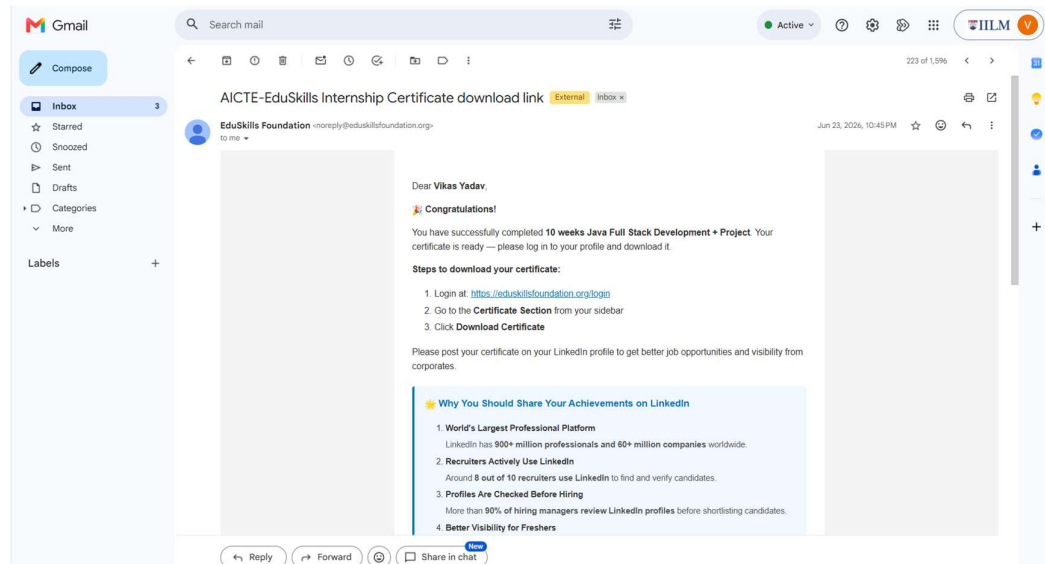
Future Enhancements

- Wire the existing Razorpay order-creation endpoint into the checkout UI to complete real payment capture and server-side payment verification (signature validation).
- Persist the shopping cart server-side for signed-in users so it survives page reloads and device changes, mirroring the wishlist pattern.
- Add advanced search capabilities (e.g., full-text or fuzzy search) beyond the current case-insensitive "contains" match on product name.

- Introduce a personalised recommendation feature building on the existing recently-viewed and review data.
- Add order-status tracking and notifications (e.g., order confirmation emails) once real payment capture is in place.
- Build a complete administrative dashboard around the existing product create/update/delete API endpoints.
- Introduce an automated test suite (unit tests for utilities such as getProductImage, integration tests for API routes) to replace the current manual test process described in Section 4.8.8.
- Add basic analytics (e.g., using the already-installed recharts library) for order volume and product performance, consistent with dependencies already present in package.json but not yet used for this purpose in the reviewed code.

4.10 Certificates of Completion and Other Communication Mail Proof

The official Certificate of Virtual Internship issued by the Ministry of Education (AICTE), the National Internship Portal and EduSkills Academy has already been presented in Chapter 3 (Internship Completion Certificate) of this report, including the certificate ID, student ID and grade awarded. The candidate should attach any additional proof of communication (mentor emails, portal notifications, evaluation communication) at the placeholders below before final submission.



5. BIBLIOGRAPHY / REFERENCES

ShopFusion Source Repository. GitHub — vikasyadav1307/shopfusion. Retrieved 2026.

Vercel Inc. Next.js Documentation — App Router. <https://nextjs.org/docs>

Meta Platforms, Inc. React Documentation. <https://react.dev>

Microsoft Corporation. TypeScript Documentation. <https://www.typescriptlang.org/docs>

Prisma Data, Inc. Prisma ORM Documentation. <https://www.prisma.io/docs>

PostgreSQL Global Development Group. PostgreSQL Documentation. <https://www.postgresql.org/docs>

Supabase Inc. Supabase Documentation. <https://supabase.com/docs>

Clerk Inc. Clerk Documentation — Next.js Integration and Middleware. <https://clerk.com/docs>

Razorpay Software Pvt. Ltd. Razorpay Orders API Documentation. <https://razorpay.com/docs>

Tailwind Labs Inc. Tailwind CSS Documentation. <https://tailwindcss.com/docs>

Poimandres / pmndrs. Zustand Documentation. <https://github.com/pmndrs/zustand>

AICTE — EduSkills National Internship Portal. Java Full Stack Development With Project — Virtual Internship Program, April–June 2026.